



Laboratorio No.1 – Semana 14 - Integración servicios en la nube

Nombre: Sebastian Alexander Villeda Reyna **Carné:** 1032625

Objetivo: Desarrollar y publicar una aplicación web que permita interactuar con archivos almacenados en Azure Blob Storage, utilizando control de versiones con Git y una canalización CI/CD de Azure DevOps para automatizar el despliegue de la aplicación en Azure App Service

Resultado esperado

- Aplicación web funcional ejecutándose en Azure App Service.
- Conexión entre App Service y Azure Blob Storage.
- Listado y consulta de archivos almacenados en un contenedor Blob.
- Operación de carga de archivos al Blob Storage.
- Repositorio Git con historial de cambios.
- Pipeline de Azure DevOps que compile y despliegue automáticamente la aplicación.
- Evidencias que demuestren el funcionamiento de la solución.

Repositorio recomendado

Utilice como punto de partida el ejemplo oficial de Microsoft/Azure Samples revisado para esta actividad:

GitHub: <https://github.com/Azure-Samples/serviceconnector-webapp-storageblob-dotnet>

El tutorial oficial utiliza una aplicación web .NET conectada a Azure Blob Storage mediante una identidad administrada y Service Connector. La documentación indica además que la aplicación puede desplegarse en Azure App Service.

Tecnologías

Tecnología	Uso
C# / .NET	Desarrollo de la aplicación web.
Azure Blob Storage	Almacenamiento de archivos.
Azure App Service	Hospedaje de la aplicación.
Git	Control de versiones.
Azure DevOps Pipelines	Integración y despliegue continuo.
Managed Identity / Service Connector	Acceso seguro desde App Service al almacenamiento.

Realización de la actividad

1. Preparar el repositorio

En el repositorio brindado arriba, realice un fork del repositorio base o copie el proyecto a un repositorio propio. El repositorio debe pertenecer al equipo de trabajo. Realice un primer commit identificable



2. Revisar la aplicación:

Ejecute la aplicación localmente y revise cómo obtiene acceso al Blob Storage. Identifique el punto del código donde se realiza la lectura/escritura de blobs y documente brevemente su funcionamiento. Visual Studio -> carpeta -> MIA_Semana14 -> git clone [repo url] -> dotnet commands

NOTA: cambia ren .csproj file -> <TargetFramework>net9.0</TargetFramework>



The screenshot shows the Visual Studio Code interface with a project named 'WebStorageSample.csproj'. The terminal output shows the command 'dotnet build' and the resulting build output. The output indicates that the project was built successfully with 11 warnings. The warnings are related to security vulnerabilities in the 'Azure.Identity' and 'Azure.Storage.Blobs' packages. The warnings are as follows:

- Warning NU1903: El paquete "Azure.Identity" 1.4.1 tiene una vulnerabilidad de gravedad alta conocida, <https://github.com/advisories/GHSA-5mfx-4wxc-rv27>
- Warning NU1902: El paquete "Azure.Identity" 1.4.1 tiene una vulnerabilidad de gravedad moderada conocida, <https://github.com/advisories/GHSA-m5vv-6r4h-3vj9>
- Warning NU1902: El paquete "Azure.Identity" 1.4.1 tiene una vulnerabilidad de gravedad moderada conocida, <https://github.com/advisories/GHSA-wvxc-855f-jrvj>
- Warning NU1902: El paquete "Azure.Identity" 1.4.1 tiene una vulnerabilidad de gravedad moderada conocida, <https://github.com/advisories/GHSA-wvxc-855f-jrvj>
- Warning NU1902: El paquete "Azure.Identity" 1.4.1 tiene una vulnerabilidad de gravedad moderada conocida, <https://github.com/advisories/GHSA-wvxc-855f-jrvj>
- Warning NU1902: El paquete "Azure.Identity" 1.4.1 tiene una vulnerabilidad de gravedad moderada conocida, <https://github.com/advisories/GHSA-wvxc-855f-jrvj>
- Warning CA2200: Si inicia de nuevo la excepción detectada, se cambia la información de la pila. (<https://learn.microsoft.com/dotnet/fundamentals/code-analysis/quality-rules/ca2200>)
- Warning CA2200: Si inicia de nuevo la excepción detectada, se cambia la información de la pila. (<https://learn.microsoft.com/dotnet/fundamentals/code-analysis/quality-rules/ca2200>)
- Warning CA2024: No usar 'StreamReader.EndOfStream' en un método asíncrono (<https://learn.microsoft.com/dotnet/fundamentals/code-analysis/quality-rules/ca2024>)

The build output also shows the compilation of the project into a DLL file.

3. Crear DotNet Env

- En terminal, instalar: **dotnet add package DotNetEnv**
- Crear .env file en raíz de proyecto, valores ->
AZURE_STORAGEBLOB_RESOURCEENDPOINT=https://miarchivos2026.blob.core.windows.net/
AZURE_STORAGE_CONTAINER_NAME=default
- Crear .env.production con los mismos valores en el mismo lugar
- Anadir cambio brindado en program.cs



e) dotnet clean -> dotnet restore -> dotnet build -> dotnet run

The screenshot displays the Microsoft Azure portal interface for the 'miaazurestorage2026savilleda' resource group. The 'Service Connector' page is active, showing a table with one connection: 'Storage - Blob' with resource name 'miastorageavilleda2026' and connection name 'storageblob_savr'. The connection status is 'Unvalidated'.

Below the Azure portal, the Visual Studio IDE is shown with the 'Program.cs' file open. The code is as follows:

```
1 using DotNetEnv;
2
3 using Microsoft.AspNetCore.Hosting;
4
5 using Microsoft.Extensions.Hosting;
6
7 namespace WebStorageSample
8 {
9
10
11     0 references
12     public class Program
13     {
14
15     0 references
16     public static void Main(string[] args)
```

The terminal output shows the command 'dotnet run' being executed. The output includes a warning about the ASP.NET Core developer certificate, followed by information about the application's listening ports (https://localhost:5001 and http://localhost:5000) and the content root path.

4. Crear recursos en Azure

En azure portal site, cree o utilice un Resource Group, una Storage Account con Blob Storage y un contenedor. El contenedor deberá tener al menos tres archivos de prueba, Puede utilizar el creado en semana 13.

5. Conectar App Service con Blob Storage



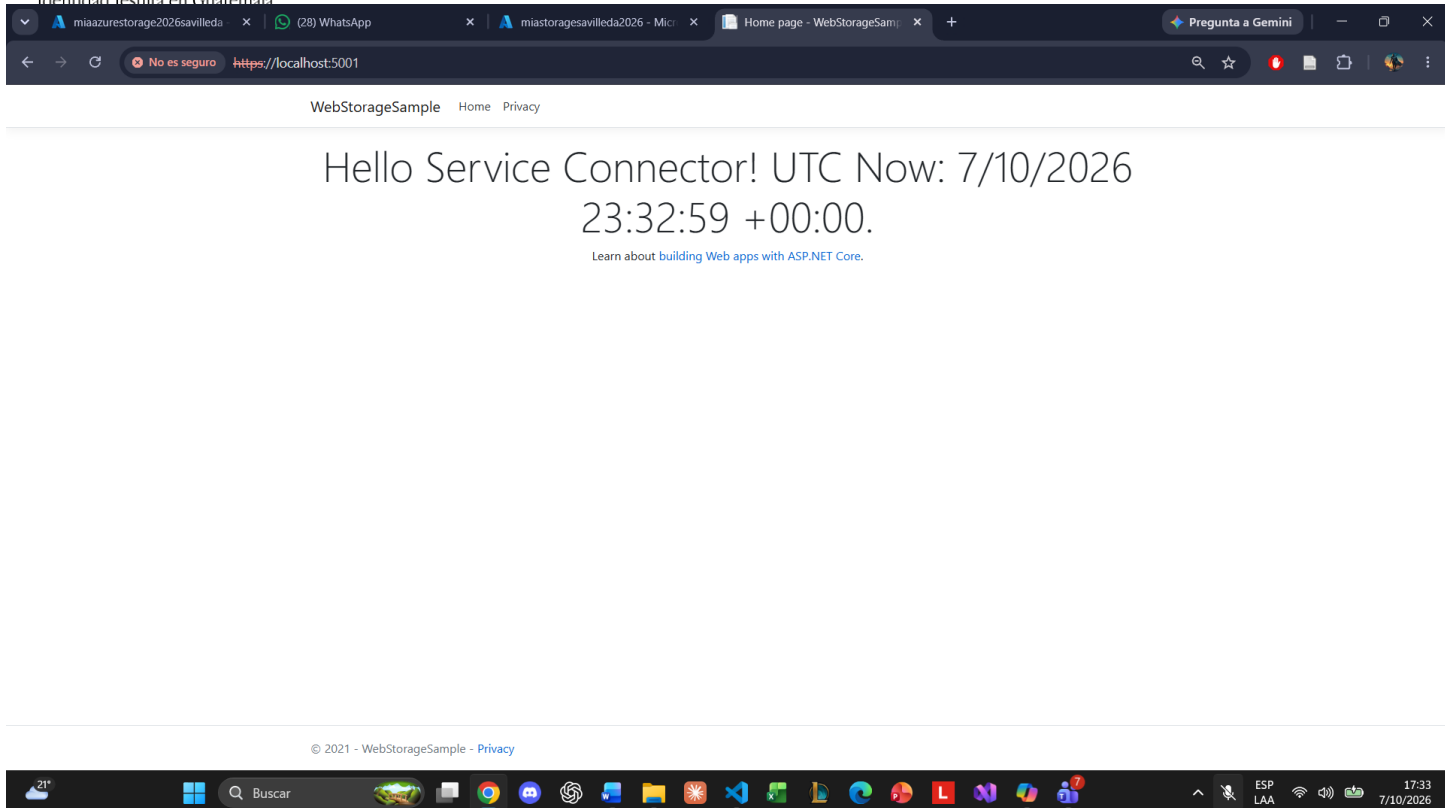
Configure la conexión entre App Service y Blob Storage mediante Service Connector utilizando identidad administrada.
Verifique que la identidad tenga los permisos necesarios sobre los blobs.

- Azure Portal → **Storage accounts** → **tu Storage Account** → **Data storage** → **Containers**
- Crear contenedor "default"
- Activa identidad administrada en App Service: **Azure Portal** → **App Service** → **Settings** → **Identity** → **System assigned: on** → **Save (web app creada semana 13)**
- Configurar la conexión entre App Service y Blob Storage: **App Service** → **Settings** → **Service Connector** → **Create** → **Seleccionar "Storage Blob"** → **Seleccionar Storage Account [tu_blob_name]** → **Authentication "System-assigned managed identity"** → **Review + Create**.
- Reemplazar storage blob endpoint y container nombre en .env file
Storage Account → **Settings** → **Endpoints** → **Blob service**

The screenshot shows the Visual Studio Code interface. The Explorer sidebar on the left displays the project structure, including files like .env, .env.production, Program.cs, and various configuration files. The main editor area shows the .env.production file with the following content:

```
1 AZURE_STORAGEBLOB_RESOURCEENDPOINT=https://miastoragesavilleda2026.blob.core.windows.net/  
2  
3 AZURE_STORAGE_CONTAINER_NAME=default
```

Below the editor, the Terminal panel is open, showing the output of the command `dotnet run`. The output includes a warning about the ASP.NET Core developer certificate and several informational messages from Microsoft.Hosting.Lifetime, indicating that the application is listening on `https://localhost:5001` and `http://localhost:5000`, and that the application has started and is now ready to accept connections.



6. Probar el acceso al almacenamiento:

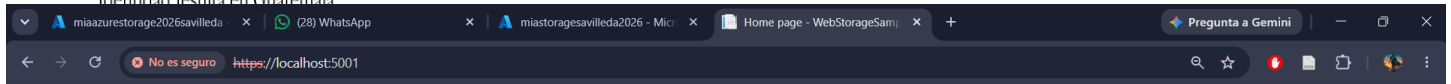
Ejecute la aplicación local y compruebe que puede leer los blobs. Realice al menos una operación de carga de archivo y verifique desde Azure Portal que el archivo llegó al contenedor. Visual Studio -> dotnet run

- Verifique que Azure CLI esté instalado ejecutando en la terminal de VS Code: `az --version`
- Inicie sesión en Azure: `az login`
- Verifique la cuenta de Azure activa: `az account show`
- Verifique que pueda obtener un token para Azure Storage: `az account get-access-token --resource https://storage.azure.com/`
- Si se muestra un accessToken la autenticación esta correcta, ejecutar -> `dotnet run`
- Si hay error en acceso, ejecutar en terminal de vs code: `az role assignment create --assignee "TU_CORREO_DE_AZURE" --role "Storage Blob Data Contributor" --scope "/subscriptions/TU_SUBSCRIPTION_ID/resourceGroups/TU_RESOURCE_GROUP/providers/Microsoft.Storage/storageAccounts/TU_STORAGE"`

Obtener info: `az account show --query "{subscriptionId:id, user:user.name}" -o table`

`az storage account show --name "tu_storage" --query "{id:id, resourceGroup:resourceGroup}" -o table`

- Cerrar process `ctr +c` y logear de nuevo: `az logout` y `az login`, Por ultimo `dotnet run`



WebStorageSample Home Privacy

Hello Service Connector! UTC Now: 7/10/2026
23:32:59 +00:00.

[Learn about building Web apps with ASP.NET Core.](#)

© 2021 - WebStorageSample - Privacy



Nota: no olvidar subir cambios a repo github fork

7. Crear Pipeline Yml

En la raíz de tu fork de GitHub crear archivo "azure-pipelines.yml"

Colocar en el archivo lo siguiente:

trigger:

- main

pool:

vmImage: 'windows-latest'

variables:

buildConfiguration: 'Release'

dotnetVersion: '9.x'

Completar posteriormente

azureSubscription: 'NOMBRE-DE-TU-SERVICE-CONNECTION'

appName: 'NOMBRE-DE-TU-APP-SERVICE'

steps:

1. Instalar .NET 9

- task: UseDotNet@2

displayName: 'Instalar .NET 9 SDK'

inputs:

packageType: 'sdk'



2. Restaurar dependencias

```
- task: DotNetCoreCLI@2
  displayName: 'Restaurar dependencias'
  inputs:
    command: 'restore'
    projects: '**/*.csproj'
```

3. Configurar ambiente de producción

```
- powershell: |
  Copy-Item ".env.production" ".env" -Force
  displayName: 'Configurar .env de producción'
```

4. Compilar

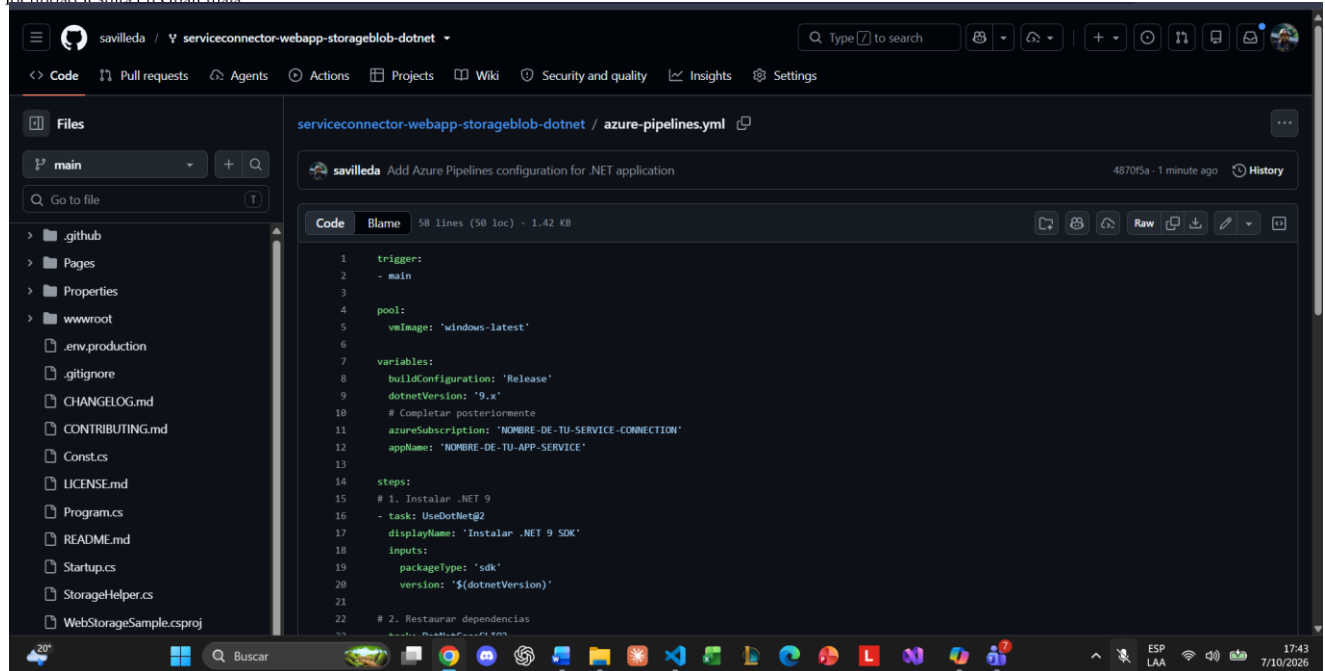
```
- task: DotNetCoreCLI@2
  displayName: 'Compilar aplicación'
  inputs:
    command: 'build'
    projects: '**/*.csproj'
    arguments: '--configuration $(buildConfiguration) --no-restore'
```

5. Publicar

```
- task: DotNetCoreCLI@2
  displayName: 'Publicar aplicación'
  inputs:
    command: 'publish'
    projects: '**/*.csproj'
    arguments: '--configuration $(buildConfiguration) --output $(Build.ArtifactStagingDirectory)/publish --no-build'
  zipAfterPublish: true
```

6. Desplegar en Azure App Service

```
- task: AzureWebApp@1
  displayName: 'Desplegar en Azure App Service'
  inputs:
    azureSubscription: '${azureSubscription}'
    appType: 'webApp'
    appName: '${appName}'
    package: '$(Build.ArtifactStagingDirectory)/publish/**/*.zip'
```



8. Crear pipeline CI/CD

Cree un pipeline YAML que restaure dependencias, compile, publique y despliegue la aplicación en Azure App Service.

- Conectarse a Azure DevOps son misma cuenta que azure portal -> <https://aex.dev.azure.com/me>
- Crear pipeline: En Azure devOps -> Menu izquierda -> pipelines -> Create Pipeline -> Seleccionar Github -> Autoriza Azure DevOps para acceder a GitHub -> seleccionar repositorio que hizo fork -> seleccionar Existing Azure Pipelines YAML file -> Branch: main Path: /azure-pipelines.yml
- Probar seleccionando "run"
- Conectar Azure DevOps con Azure: Project Settings → Service connections → New service connection -> Azure Resource Manager -> luego, Workload identity federation -> Selecciona tu suscripción de Azure -> y resource group donde estan tus recursos -> Anadir nombre "MIA_Connection" -> Seleccionar "grant access permission to all pipelines" -> Save



Description	Stages	Status	Time
#20261008.3 • Se agregan configuraciones y cambios de semana 14 Individual CI for main baac9781	✓	Success	4m ago 2m 34s
#20261008.2 • Se agregan configuraciones y cambios de semana 14 Individual CI for main 7e7b5997	✓	Success	27m ago 3m 13s
#20261008.1 • Update azure-pipelines.yml Individual CI for main a5f7ba9c	✗	Failed	44m ago 1m 33s
#20261007.1 • Add Azure Pipelines configuration for .NET application Manually run by main 4870f5ac	✗	Failed	56m ago <1s

8. Conectar Pipeline con App

- Obtener nombre app: En Azure Portal App Services → tu aplicación → Copia nombre app
- en github en tu "azure-pipelines.yml" file busca: azureSubscription y appName, cambiar por valores reales.

azureSubscription: TU_Service_Connection_name

appName: Tu_app_name

- Al hacer commit, verificar el pipeline se ejecuta automáticamente



9. Validar la solución

Compruebe desde la URL pública que la aplicación funciona y que el acceso al Blob Storage continúa operativo después del despliegue.

- acceso de la aplicación al Blob: Azure Portal → App Service → Settings → Environment variables → App settings
-> colocar `AZURE_STORAGEBLOB_RESOURCEENDPOINT`
https://tu_storage.blob.core.windows.net/
(si existe obviar)
- Verificar app: Azure Portal → App Service → OverView → Default Domain → click en URL publica
- En la app publicada realizar cambios en archivos
- Verificar que los cambios se representen en storage account: Azure Portal → Storage Account → Storage Browser → Blob containers → Seleccionar su container → Verificar cambios



Subir Imagen a Azure Blob Storage

Imagen recuperada del Blob Storage:

© 2021 WebStaging - Privacy

10. Preguntas de analisis

- ¿Para qué sirve el archivo .env en la aplicación?
Almacena variables de entorno locales (como endpoints de almacenamiento y nombres de contenedores). Permite desacoplar las configuraciones dependientes del entorno del código fuente, evitando exponer datos sensibles o rutas específicas en el control de versiones de Git.
- ¿Por qué utilizar Blob Storage y no guardar los archivos dentro del proyecto?
Para garantizar persistencia, escalabilidad y rendimiento. Los servidores de App Service en la nube ejecutan instancias sin estado (stateless); si los archivos se guardaran localmente en el servidor web, se perderían al reiniciar la aplicación, al escalar a múltiples instancias o al realizar un nuevo despliegue vía CI/CD.
- ¿Cuál es la diferencia entre el endpoint de Blob Storage y la URL de App Service?
El endpoint de Blob Storage (https://miastorageavilleda2026.blob.core.windows.net/) apunta directamente a la API del servicio de objetos persistentes de Azure. La URL del App Service (https://miaazurestorage2026avilleda-fqeshmg9a3c9e3eb.westus-01.azurewebsites.net/) es la dirección donde se hospeda e interpreta el código de la aplicación web encargada de procesar las peticiones del usuario.
- ¿Por qué utilizar una identidad administrada en lugar de una Storage Account Key?
Sigue el principio de Zero Trust y elimina la necesidad de gestionar o rotar credenciales explícitas. La Identidad Administrada (Managed Identity) permite que Azure gestione de forma nativa e invisible los tokens de autenticación entre App Service y Blob Storage sin almacenar secretos o llaves dentro del código o variables de configuración.
- ¿Qué función cumplen GitHub, Azure DevOps y la Service Connection?

GitHub: Es el repositorio de control de versiones donde reside el código fuente y las plantillas YAML de la canalización.

- Azure DevOps:** Es la plataforma de CI/CD que orquesta las tareas automatizadas de restauración, compilación, empaquetado y despliegue del proyecto.



- **Service Connection:** Es el mecanismo de autenticación segura (vía Workload Identity Federation) que le otorga permisos a Azure DevOps para desplegar recursos directamente en la suscripción de Azure Portal sin compartir credenciales de usuario

6. ¿Qué ocurre después de hacer push a main?

Se activa el disparador automatizado (trigger: - main) de azure-pipelines.yml. Azure DevOps descarga el último commit, configura el SDK de .NET, restaura dependencias, compila el proyecto, empaqueta el artefacto en ZIP y lo publica automáticamente en Azure App Service.

7. Si el pipeline termina correctamente pero la aplicación recibe un 403 al subir un archivo, ¿qué revisarías?

- Que la Identidad Administrada del App Service tenga asignado el rol **Storage Blob Data Contributor** en el Control de Acceso (IAM) del Storage Account.
- Que las variables de entorno (AZURE_STORAGEBLOB_RESOURCEENDPOINT y AZURE_STORAGE_CONTAINER_NAME) estén correctamente declaradas en la configuración de App Service.
- Que el estado del Service Connector se encuentre en estado *Validated / OK*.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-14/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.